## CS851: Network Security

# Assignment 2 Lab Report

**Submitted by**


**SAMYAK SHRIRAM GEDAM**

Roll No: 252IS032

*II Semester M-Tech CSIS*

Department of Computer Science and Engineering
National Institute of Technology Karnataka
Surathkal, Mangalore-575025, India


**2025–2026**

# Introduction

Modern enterprise networks are complex systems composed of multiple interdependent layers, each introducing unique security vulnerabilities that attackers can exploit. A security weakness at one layer can propagate and amplify risks across other layers, ultimately compromising the confidentiality, integrity, and availability (CIA) of networked systems.

The objective of this problem is to systematically analyze and evaluate vulnerabilities across different network layers—from the physical infrastructure to applications—using established models such as the OSI/TCP-IP stack, MITRE ATT&CK, and STRIDE.

## Verifying connectivity

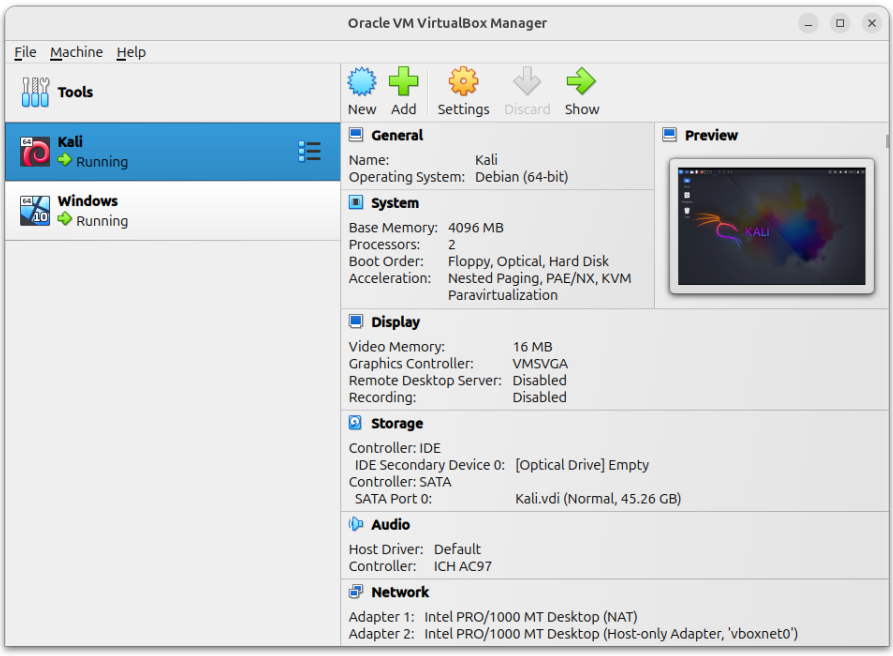



Figure 1: VirtualBox VM

Two virtual machines were configured using Oracle VirtualBox on an Ubuntu host system. **Kali Linux** was set up as the attacker machine and **Windows OS** as the victim machine. Both virtual machines were connected using an **isolated internal network** and assigned IP addresses in the same subnet. Successful bidirectional connectivity was verified using ICMP ping, confirming readiness for further security experiments."

Table 1: Virtual Machines Network Setup

| **Role** | **OS** | **IP address** |
|---|---|---|
| Attacker | Kali Linux | 192.168.56.103 |
| Victim | Windows 10 | 192.168.56.101 |
| Network | VirtualBox Host-Only Network | 192.168.56.0/24 |

Justification: Isolated testbed for safe attack simulation

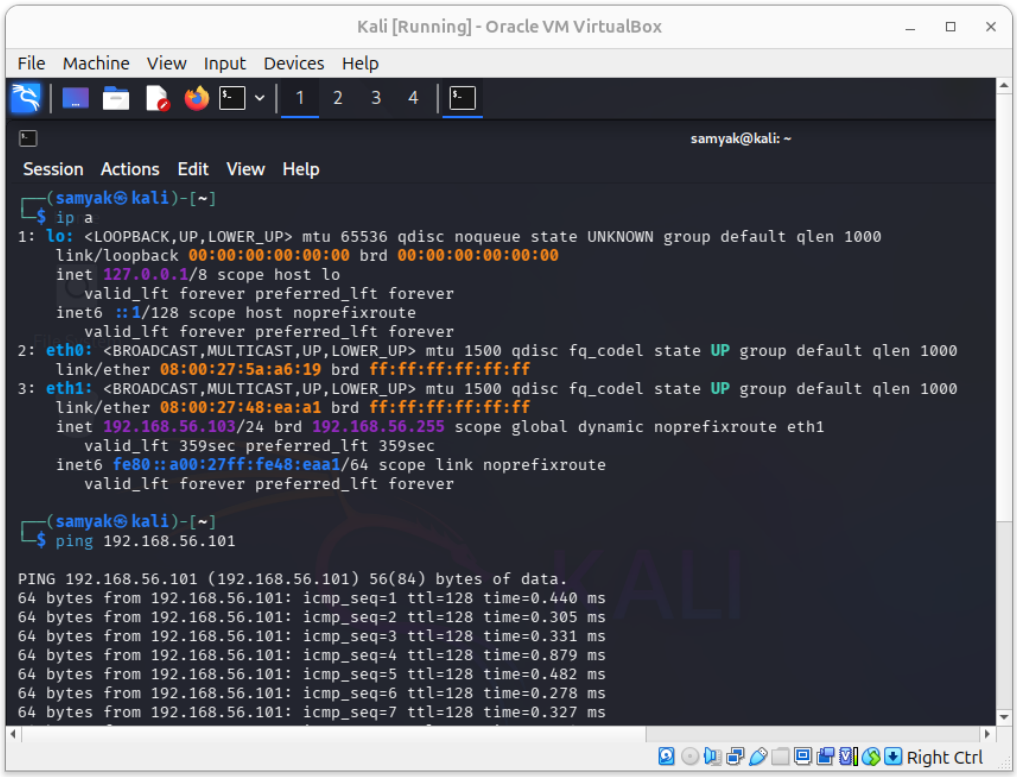



Figure 2: Ping Kali to Windows

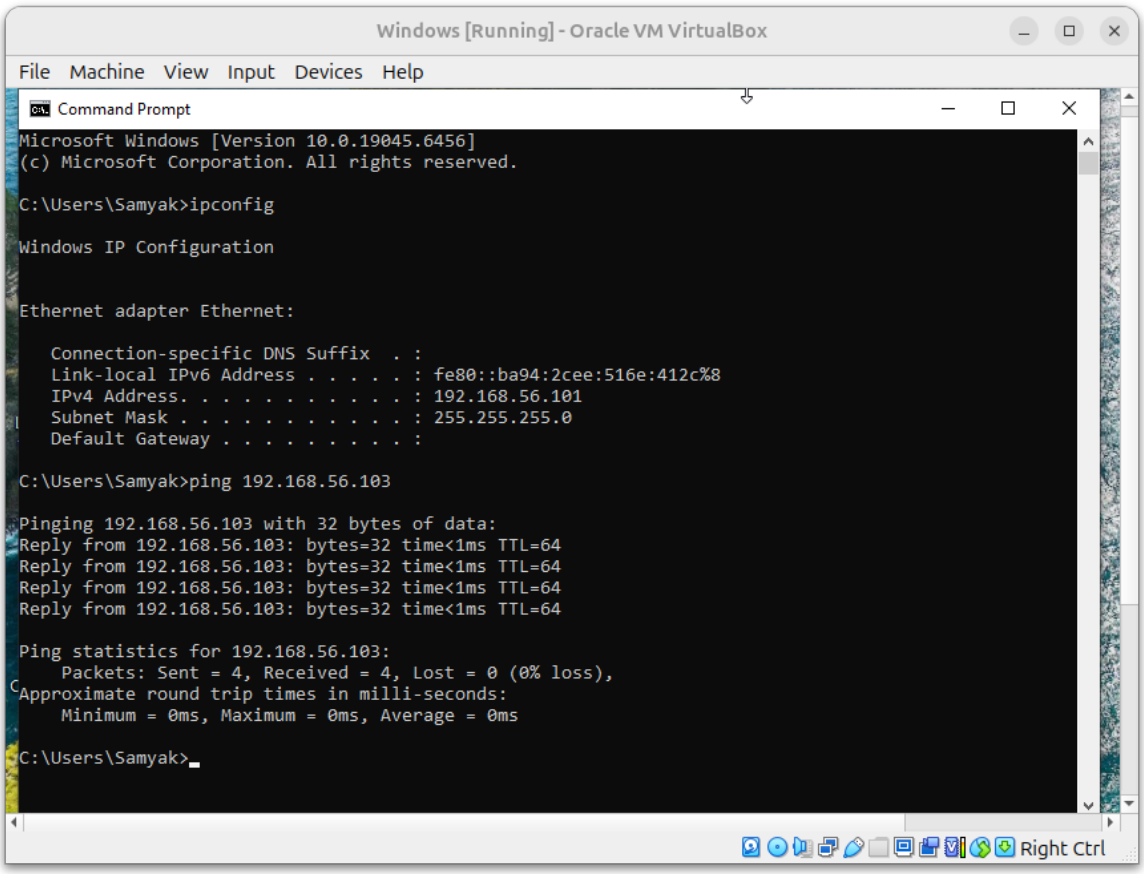



Figure 3: Ping Windows to Kali

# Layer-Wise Attacks

## Task 1: Physical / Infrastructure Layer Attack

### Attack Simulated:

Network interface disruption (link failure simulation)

### Procedure:

The Ethernet network adapter of the victim Windows virtual machine was manually disabled to simulate a physical layer fault. From the attacker machine (Kali Linux), ICMP echo requests were continuously sent to the victim to observe network behavior during the disruption.

### Observation:

After disabling the Ethernet adapter, the victim system became unreachable from the network. The attacker machine received repeated "Destination Host Unreachable" messages, indicating complete loss of connectivity.

### Impact Analysis:

Table 2: Impact on Security Attributes

| **Security Attribute** | **Impact** |
|---|---|
| Confidentiality | Not affected |
| Integrity | Not affected |
| Availability | Completely compromised |

### MITRE ATT&CK Mapping:

T1499 – Endpoint Denial of Service

### Conclusion:

This experiment demonstrates that failure or compromise at the physical or infrastructure layer directly impacts network availability and can render higher-layer services inaccessible.

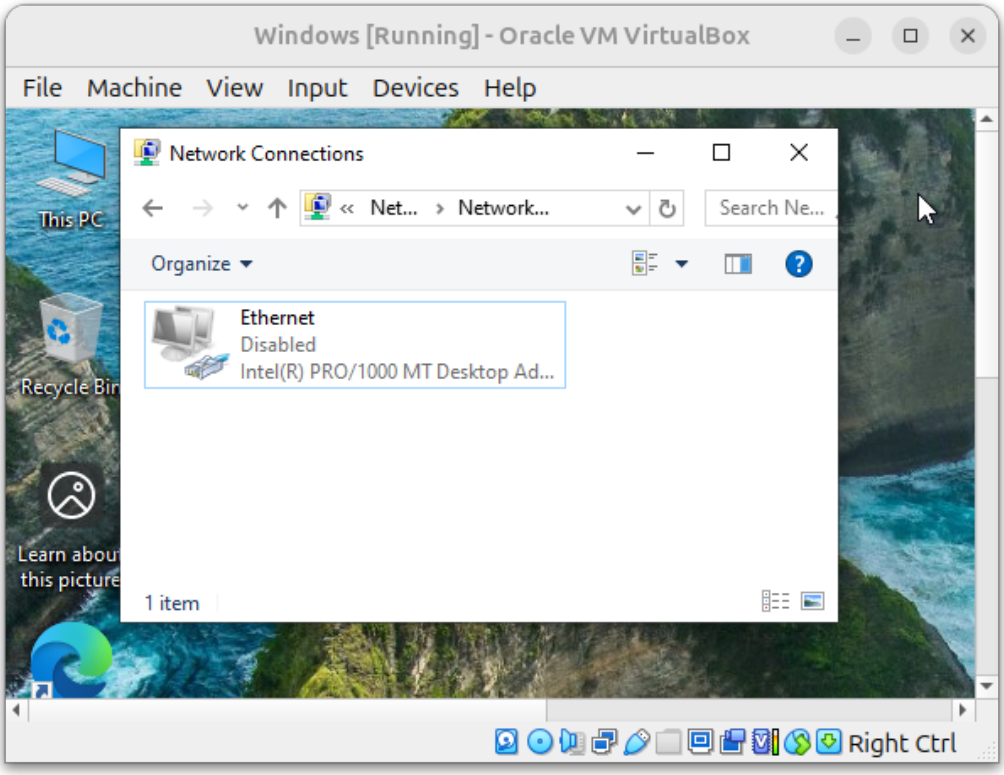



Figure 4: Disable Ethernet

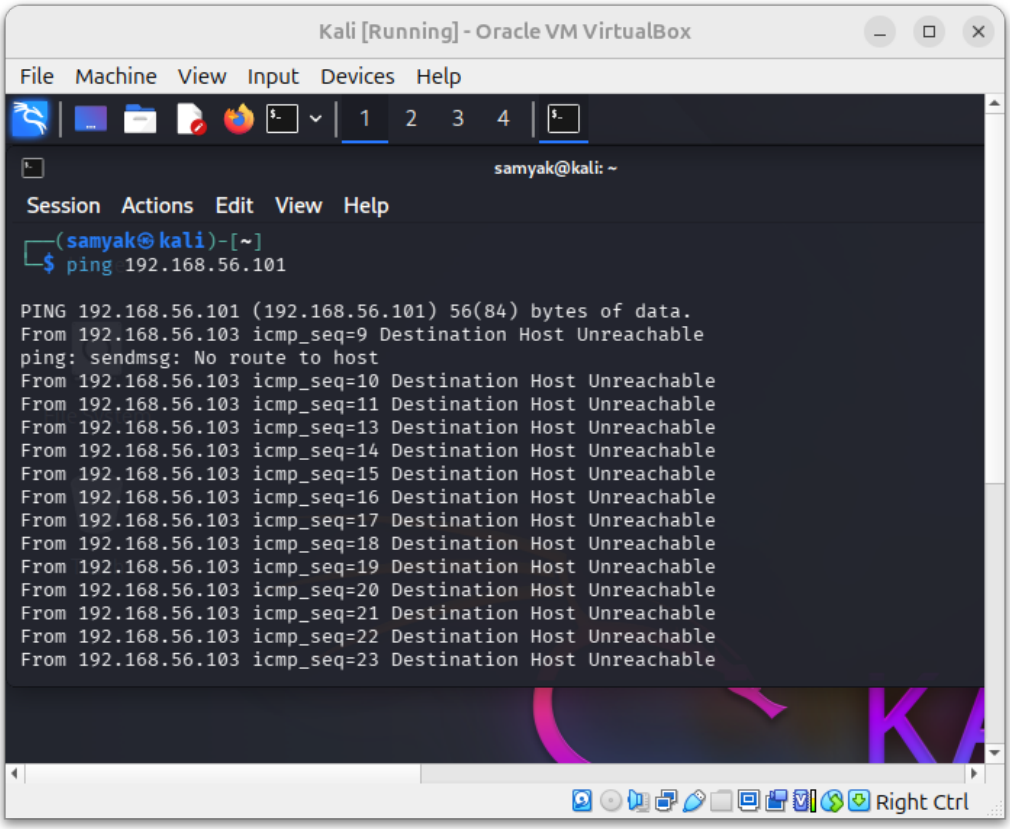



Figure 5: Ping Failure

## Task 2: Data Link Layer (Layer 2): ARP Spoofing / MITM Attack

### Identify Network Details (Kali)

Table 3: Network Configuration for Attack Scenario

| Role | IP / Interface |
|---|---|
| Kali (Attacker) | `192.168.56.103` |
| Windows (Victim) | `192.168.56.101` |
| Interface to use | `eth1` |

**Attack Performed:**

ARP Spoofing (Man-in-the-Middle attack)

**Tools Used:**

arpspoof

**Procedure:**

Before launching the attack, On the attacker machine, IP forwarding was checked and then enabled to allow packet forwarding.
The ARP cache of the victim system was examined before launching the attack using: **arp -a**
The ARP cache of the victim system was examined before launching the attack using:
**sudo arpspoof -i eth1 -t 192.168.56.101 192.168.56.100**
**sudo arpspoof -i eth1 -t 192.168.56.100 192.168.56.101**
Continuous forged ARP replies were sent by the attacker, falsely associating the attacker's MAC address with the legitimate IP addresses.
The ARP table of the victim system was examined again after the attack to observe changes.

**Oservation:**

Before the attack, the victim's ARP table showed correct IP-to-MAC address mappings for the network hosts. After launching ARP spoofing, the victim's ARP table was modified such that multiple IP addresses resolved to the attacker's MAC address. This confirms successful ARP poisoning and establishment of a Man-in-the-Middle position at the Data Link Layer.

**Impact Analysis (CIA):**

Table 4: Security Impact Analysis

| Security Attribute | Impact | Explanation |
|---|---|---|
| **Confidentiality** | Compromised | Attacker can sniff credentials, sessions, and plain-text traffic |
| **Integrity** | Compromised | Attacker can modify packets in transit |
| **Availability** | Potentially affected | If forwarding is disabled, traffic can be dropped (DoS) |

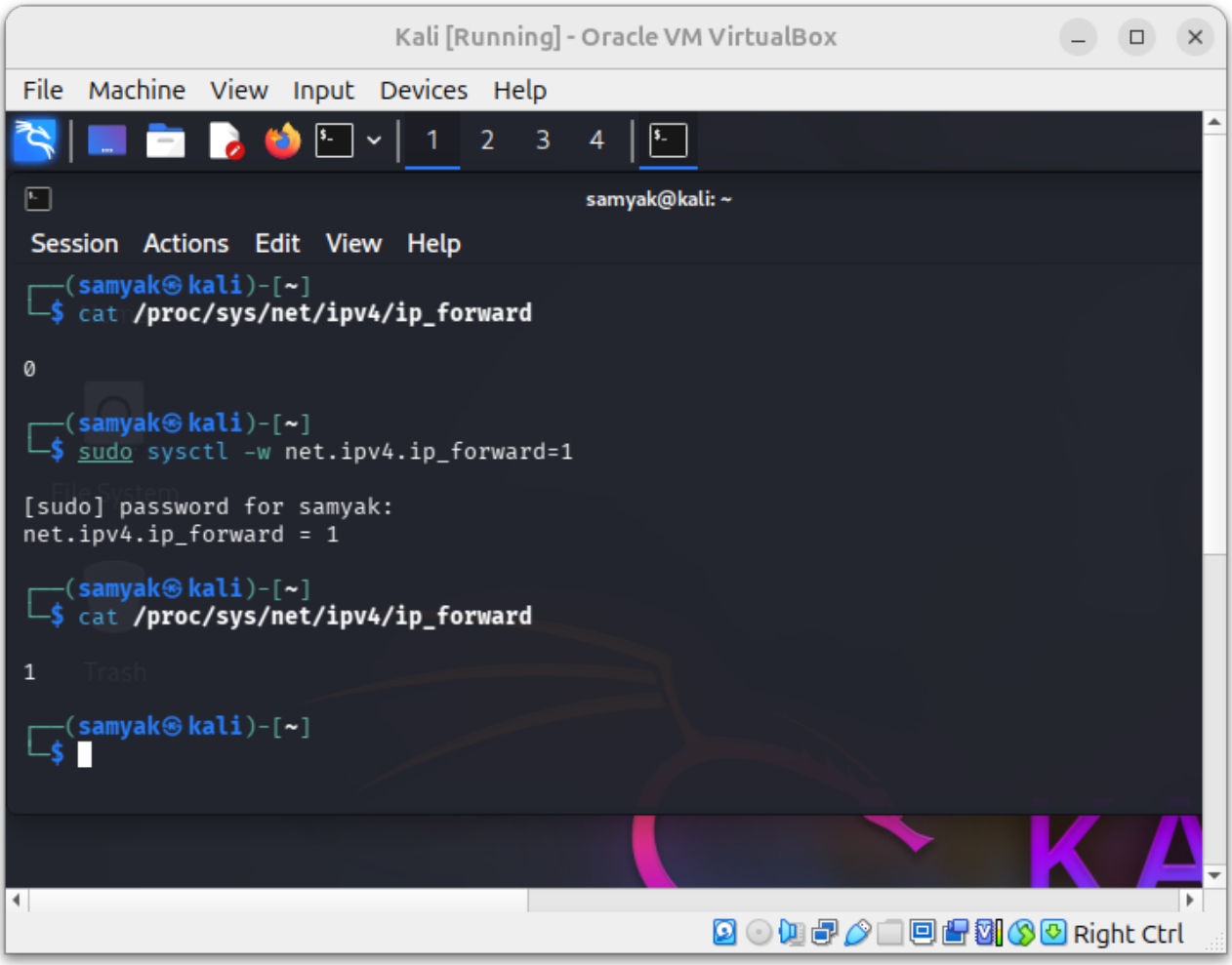



Figure 6: Enable IP Forwarding

**MITRE ATT&CK Mapping:**

T1557.002 – Adversary-in-the-Middle: ARP Cache Poisoning

**Conclusion:**

This experiment demonstrates that ARP lacks authentication mechanisms, making it vulnerable to spoofing attacks. By exploiting this weakness, an attacker can manipulate Layer 2 address resolution and position themselves as a Man-in-the-Middle, enabling interception and manipulation of network traffic.

```
C:\Windows\system32>arp -a

Interface: 192.168.56.101 --- 0x8
  Internet Address      Physical Address      Type
  192.168.56.100        08-00-27-1d-d1-72     dynamic
  192.168.56.103        08-00-27-48-ea-a1     dynamic
  224.0.0.22            01-00-5e-00-00-16     static
  239.255.255.250       01-00-5e-7f-ff-fa     static
```

Figure 7: arp - a before attack

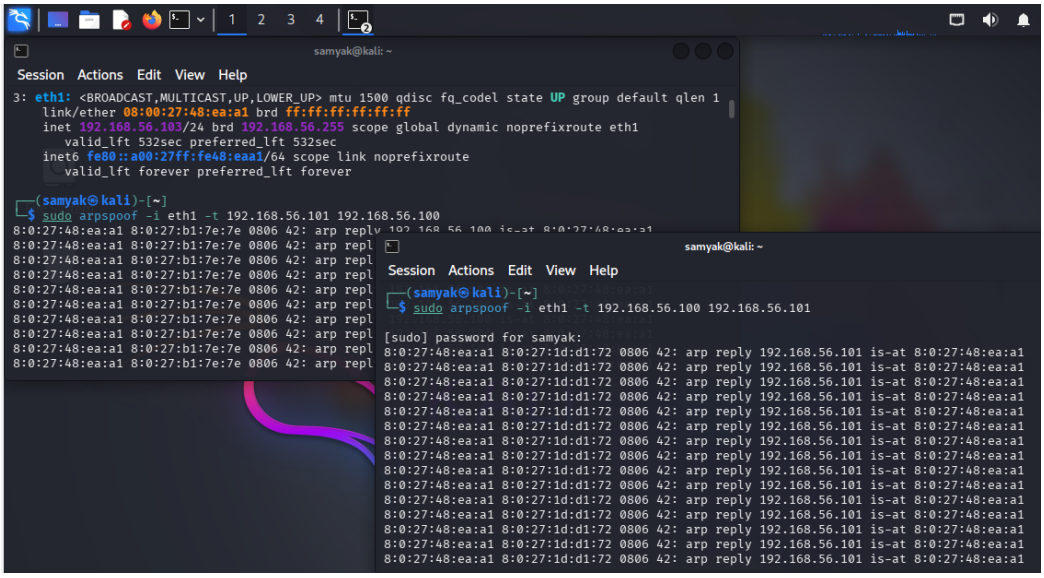



Figure 8: Launch Bidirectional ARP Spoofing

```
C:\Windows\system32>arp -a

Interface: 192.168.56.101 --- 0x8
  Internet Address      Physical Address      Type
  192.168.56.100        08-00-27-48-ea-a1     dynamic
  192.168.56.103        08-00-27-48-ea-a1     dynamic
  192.168.56.255        ff-ff-ff-ff-ff-ff     static
  224.0.0.22            01-00-5e-00-00-16     static
  224.0.0.251           01-00-5e-00-00-fb     static
  224.0.0.252           01-00-5e-00-00-fc     static
  239.255.255.250       01-00-5e-7f-ff-fa     static

C:\Windows\system32>_
```

Figure 9: arp - a after attack

## Task 3: Network Layer (Layer 3) — ICMP Flood Attack

### Attack Performed:

ICMP Flood (Network Denial of Service)

### Tool Used:

hping3

### Procedure

An ICMP flood attack was launched from the Kali Linux attacker machine using the hping3 tool. A large volume of ICMP echo request packets was sent continuously to the victim Windows system to overwhelm network resources and disrupt normal communication.

### Observation

During the attack, the victim system experienced a significant increase in network traffic, as observed in the Task Manager. Legitimate ICMP responses were delayed or dropped, resulting in packet loss and degraded network performance.

### Impact Analysis (CIA):

Table 5: Impact on Security Attributes

| **Security Attribute** | **Impact** |
|---|---|
| Confidentiality | Not affected |
| Integrity | Not affected |
| Availability | Severely compromised |

### MITRE ATT&CK Mapping:

T1498 – Network Denial of Service

### Conclusion

This experiment demonstrates that the network layer is vulnerable to flooding attacks, which can exhaust bandwidth and prevent legitimate traffic from reaching the target system, thereby impacting service availability.

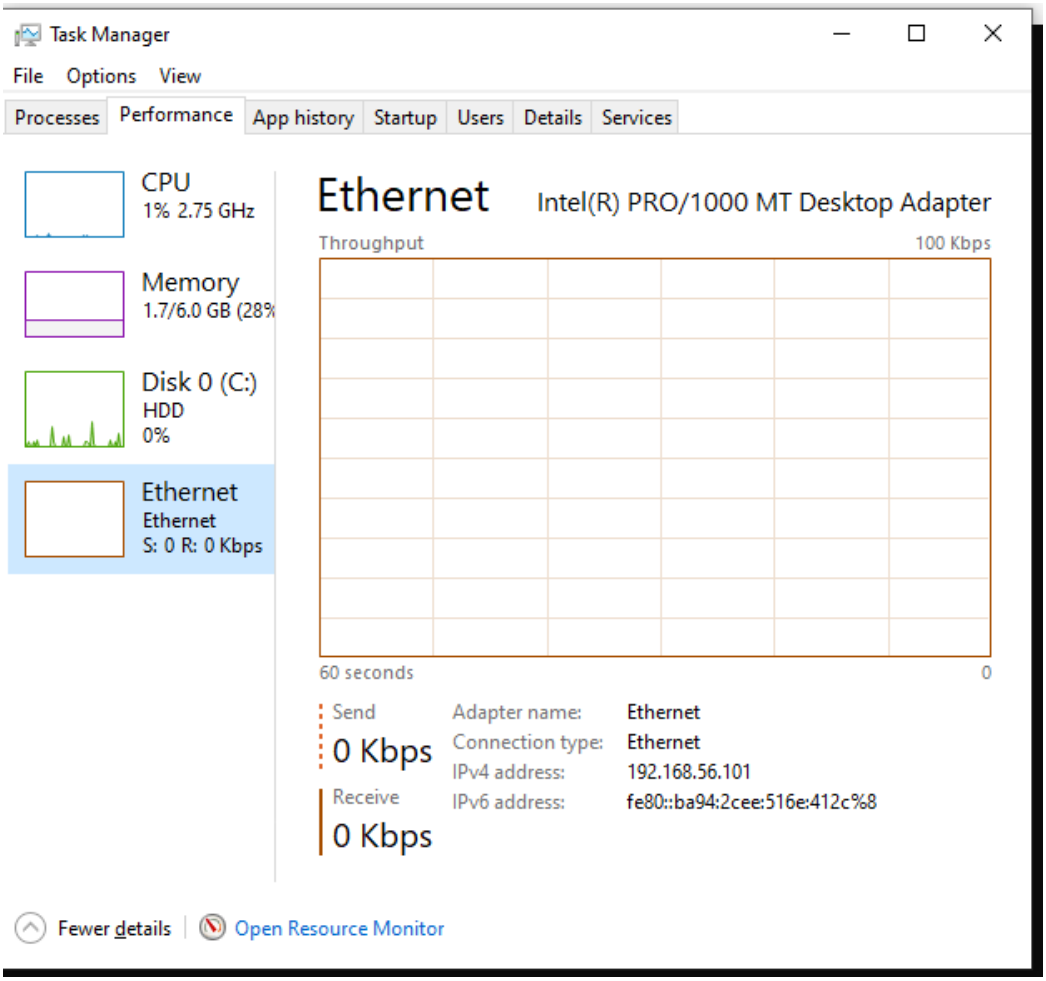



Figure 10: Before attack

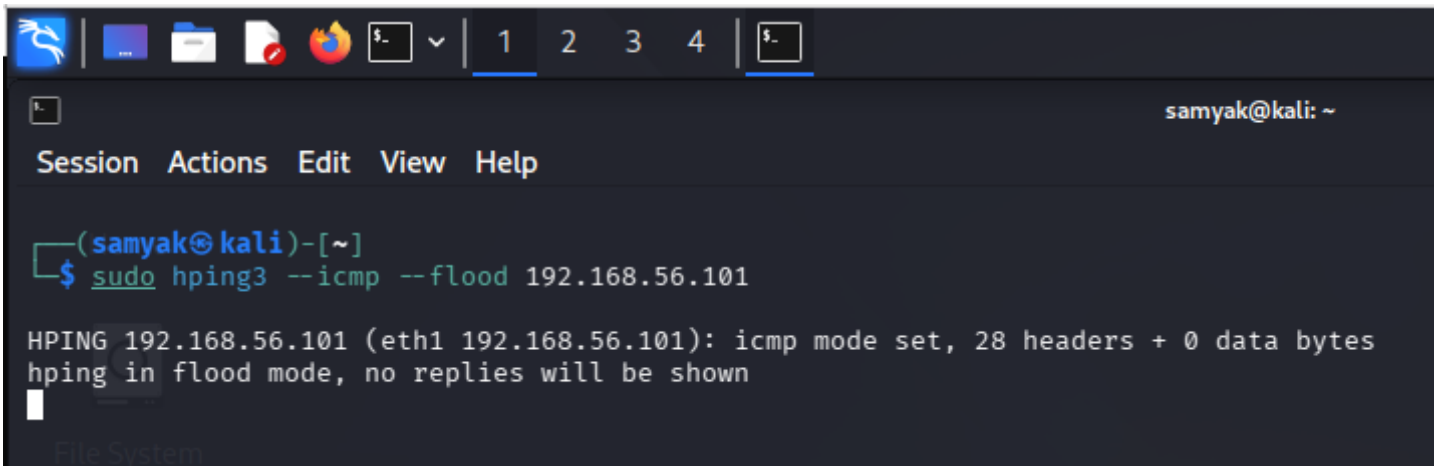



Figure 11: ICMP flooding

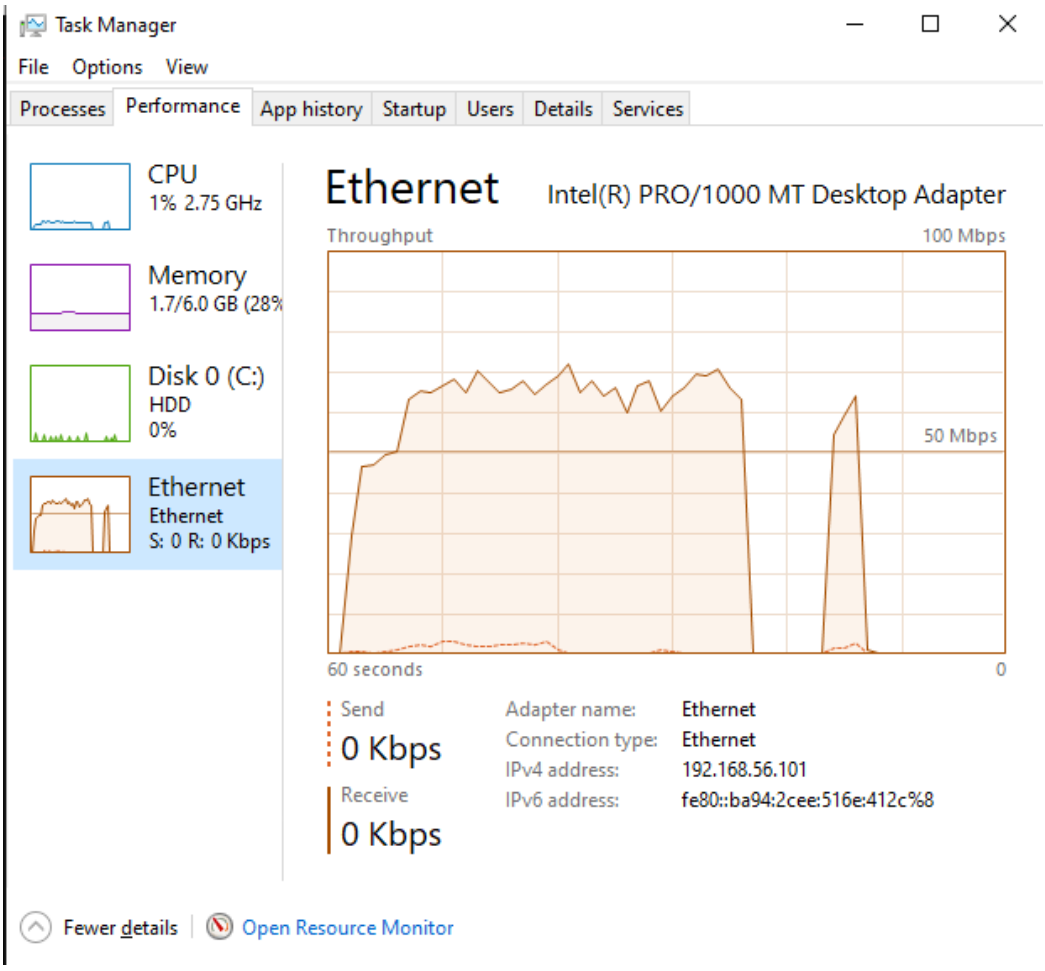



Figure 12: After attack

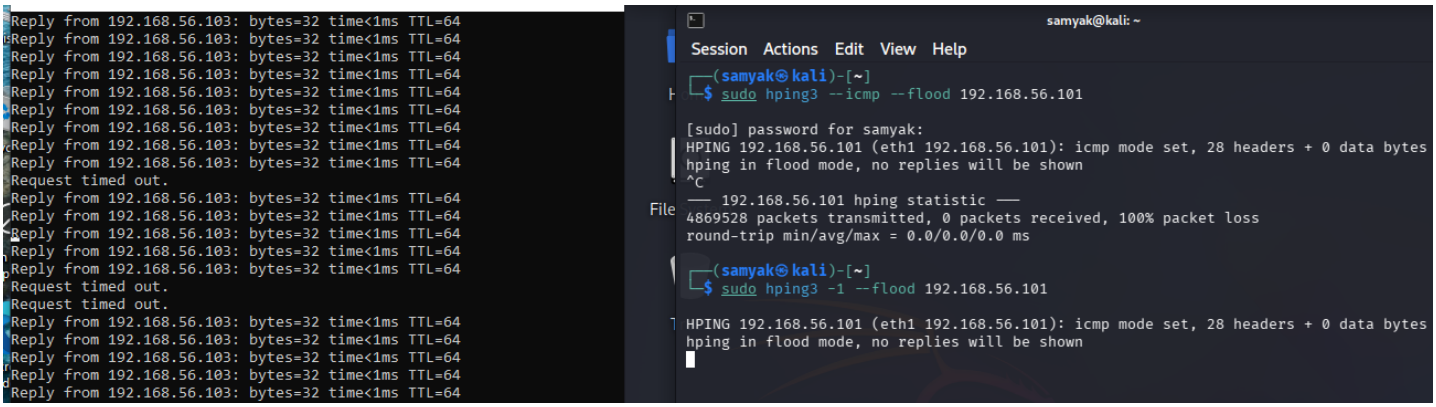



Figure 13: Packet loss

## Task 4: Transport Layer (Layer 4) — TCP SYN Flood Attack

**Attack Performed:**

TCP SYN Flood (Transport Layer Denial of Service)

**Tool Used:**

hping3

**Procedure:**

A TCP SYN flood attack was launched from the Kali Linux attacker machine using the hping3 tool. A large volume of TCP SYN packets was sent to the victim Windows system on commonly used ports (80 and 445) without completing the three-way handshake.

**Observation:**

During the attack, the victim system accumulated a large number of half-open TCP connections in the SYN_RECEIVED state, as observed using the netstat -an command. The victim was unable to efficiently process new legitimate connection requests.

**Impact Analysis (CIA):**

Table 6: Impact on Security Attributes

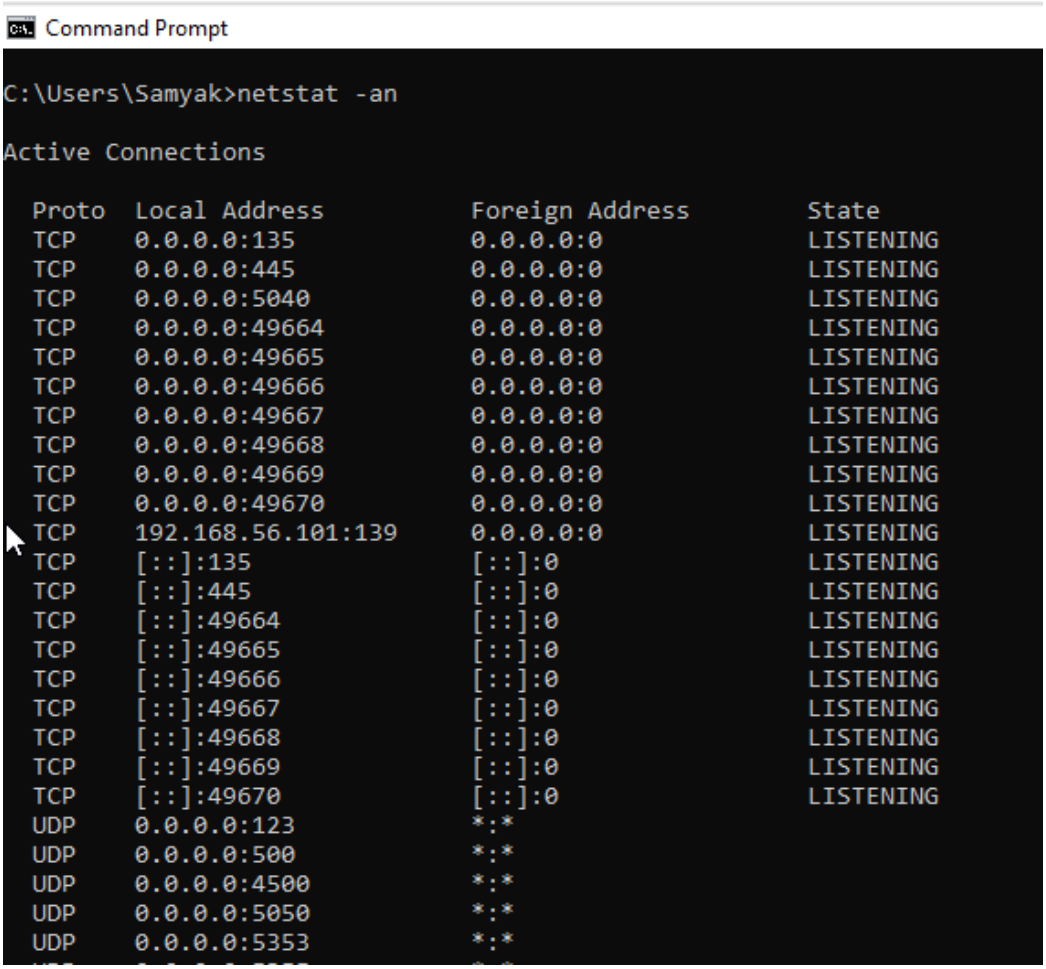
| **Security Attribute** | **Impact** |
|---|---|
| Confidentiality | Not affected |
| Integrity | Not affected |
| Availability | Severely compromised |

**MITRE ATT&CK Mapping:**

T1499 – Endpoint Denial of Service

**Conclusion:**

This experiment demonstrates that the transport layer is vulnerable to SYN flood attacks, where an attacker exploits the TCP handshake mechanism to exhaust server resources and deny legitimate access.



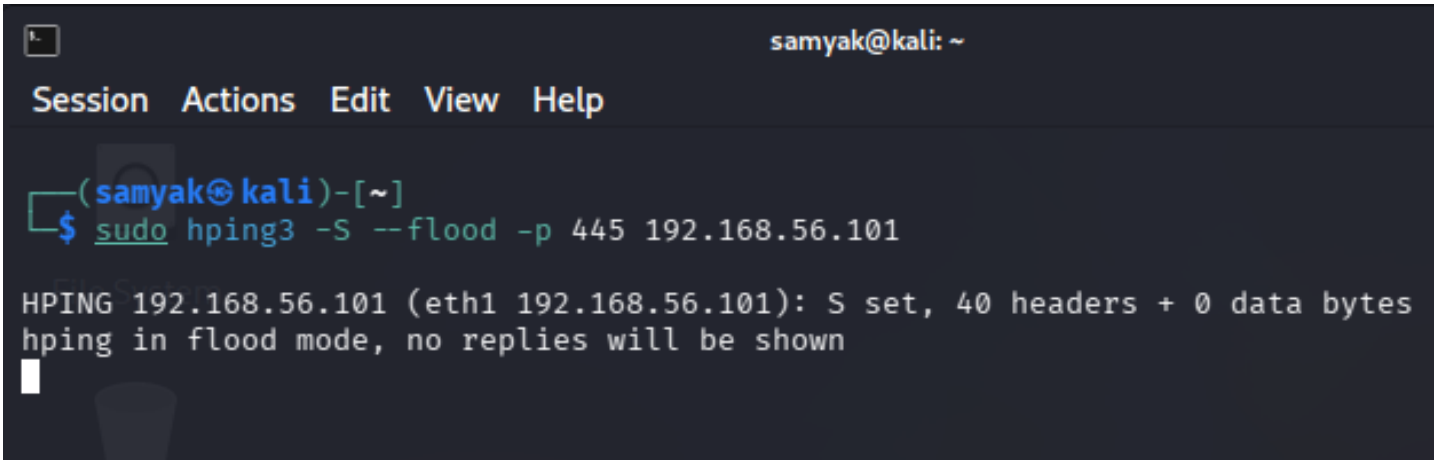
```
Command Prompt

C:\Users\Samyak>netstat -an

Active Connections

  Proto  Local Address          Foreign Address        State
  TCP    0.0.0.0:135            0.0.0.0:0              LISTENING
  TCP    0.0.0.0:445            0.0.0.0:0              LISTENING
  TCP    0.0.0.0:5040           0.0.0.0:0              LISTENING
  TCP    0.0.0.0:49664          0.0.0.0:0              LISTENING
  TCP    0.0.0.0:49665          0.0.0.0:0              LISTENING
  TCP    0.0.0.0:49666          0.0.0.0:0              LISTENING
  TCP    0.0.0.0:49667          0.0.0.0:0              LISTENING
  TCP    0.0.0.0:49668          0.0.0.0:0              LISTENING
  TCP    0.0.0.0:49669          0.0.0.0:0              LISTENING
  TCP    0.0.0.0:49670          0.0.0.0:0              LISTENING
  TCP    192.168.56.101:139     0.0.0.0:0              LISTENING
  TCP    [::]:135               [::]:0                 LISTENING
  TCP    [::]:445               [::]:0                 LISTENING
  TCP    [::]:49664             [::]:0                 LISTENING
  TCP    [::]:49665             [::]:0                 LISTENING
  TCP    [::]:49666             [::]:0                 LISTENING
  TCP    [::]:49667             [::]:0                 LISTENING
  TCP    [::]:49668             [::]:0                 LISTENING
  TCP    [::]:49669             [::]:0                 LISTENING
  TCP    [::]:49670             [::]:0                 LISTENING
  UDP    0.0.0.0:123            *:*
  UDP    0.0.0.0:500            *:*
  UDP    0.0.0.0:4500           *:*
  UDP    0.0.0.0:5050           *:*
  UDP    0.0.0.0:5353           *:*
```

Figure 14: Listening TCP before attack



```
samyak@kali: ~
Session  Actions  Edit  View  Help

┌──(samyak㉿kali)-[~]
└─$ sudo hping3 -S --flood -p 445 192.168.56.101

HPING 192.168.56.101 (eth1 192.168.56.101): S set, 40 headers + 0 data bytes
hping in flood mode, no replies will be shown
```

Figure 15: SYN flooding on port 445

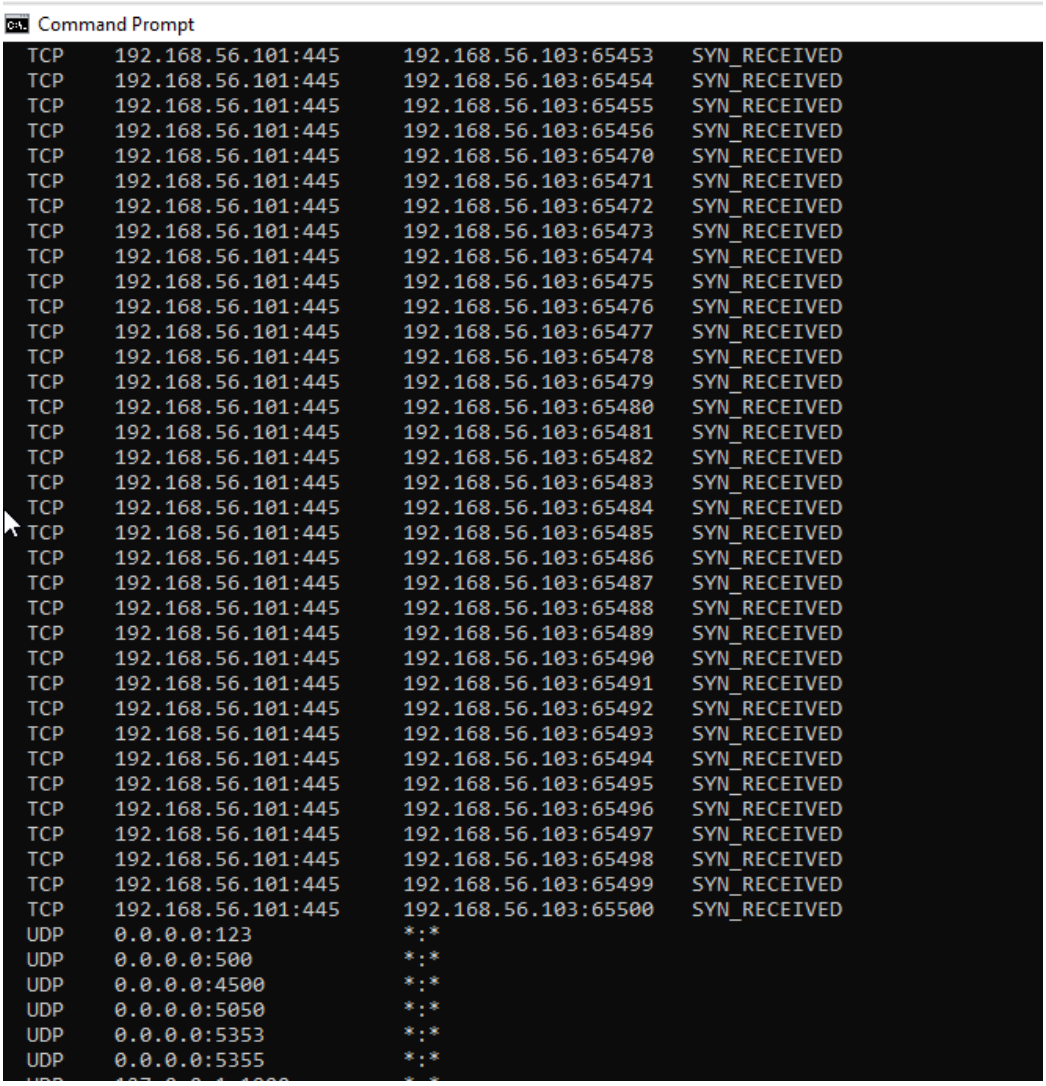



Figure 16: TCP after attack

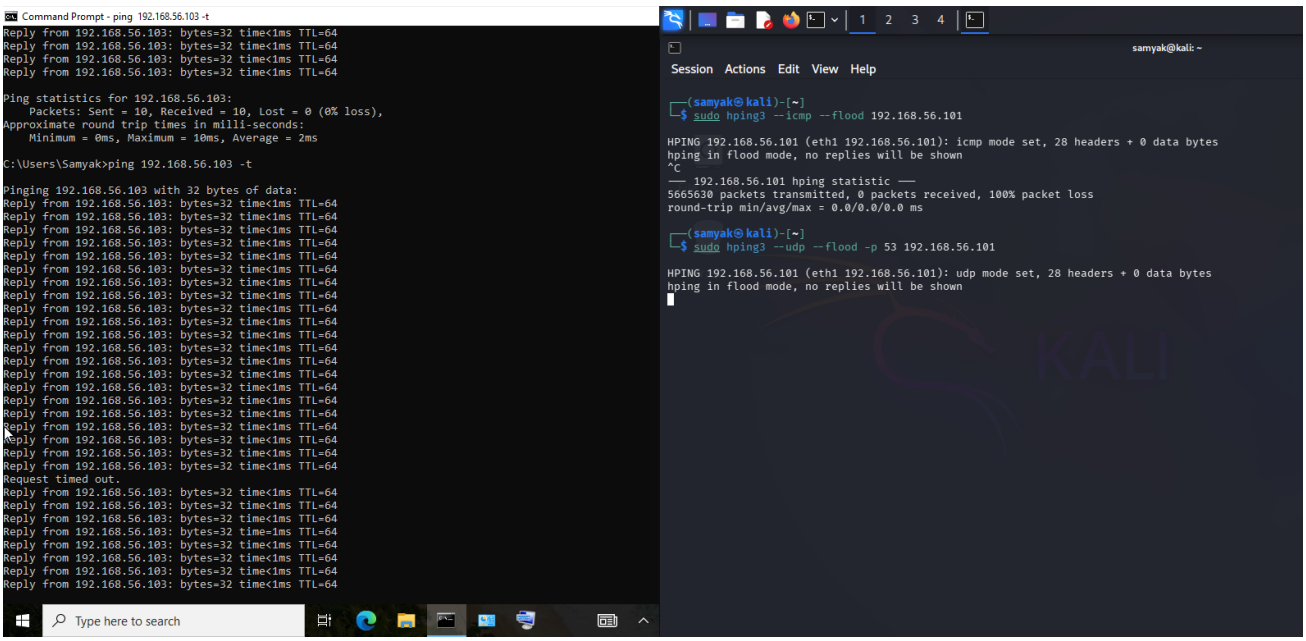



Figure 17: Udp timeout

## Task 5: Session Layer (Layer 5) — Session Hijacking

**Attack Performed:**

Session Hijacking using Man-in-the-Middle attack

**Tools Used:**

arpspoof, Wireshark

**Procedure:**

An ARP spoofing attack was launched from the attacker machine (Kali Linux) to position itself as a man-in-the-middle between the victim and the network. A local HTTP server was hosted on the attacker system, and the victim accessed the server through a web browser, establishing an active HTTP session. Network traffic was captured using Wireshark.

**Observation:**

The attacker was able to capture and observe active HTTP session traffic, including request and response packets, in clear text. This confirms that session communication was intercepted while in transit.

**Impact Analysis (CIA):**

Table 7: Impact on Security Attributes

| Security Attribute | Impact |
|---|---|
| Confidentiality | Compromised |
| Integrity | Potentially compromised |
| Availability | Not affected |

**MITRE ATT&CK Mapping:**

- T1550 – Use of Stolen Session Tokens
- T1557 – Adversary-in-the-Middle

**Conclusion:**

This experiment demonstrates that weak session handling and unencrypted communication at the session layer can allow attackers to intercept and hijack active sessions, leading to unauthorized access and information exposure.

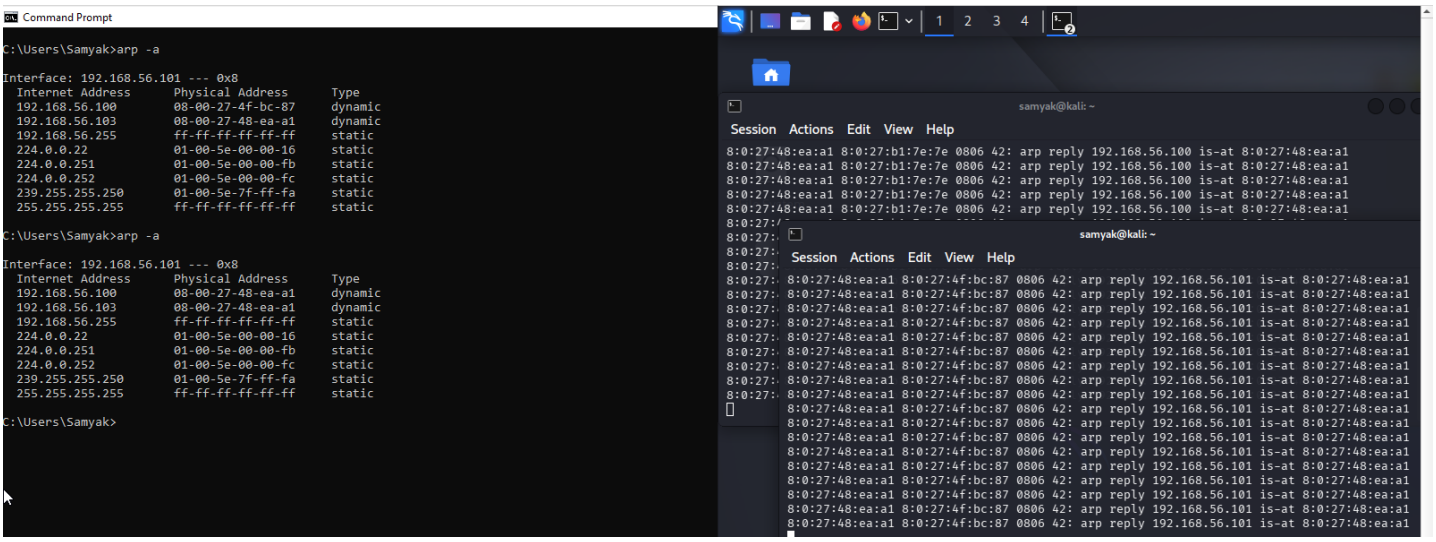



Figure 18: Create MITM

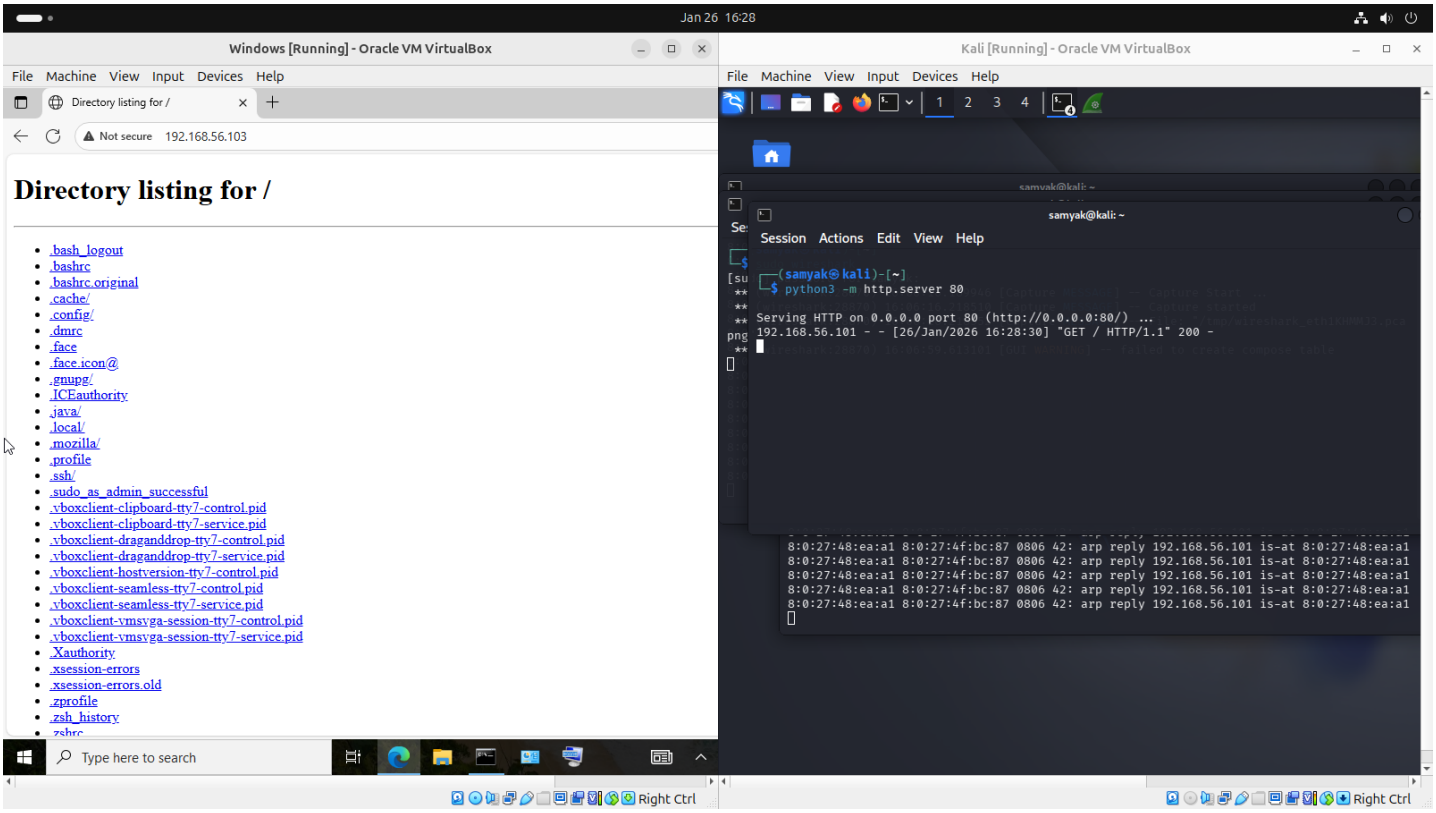



Figure 19: Establish Local HTTP session

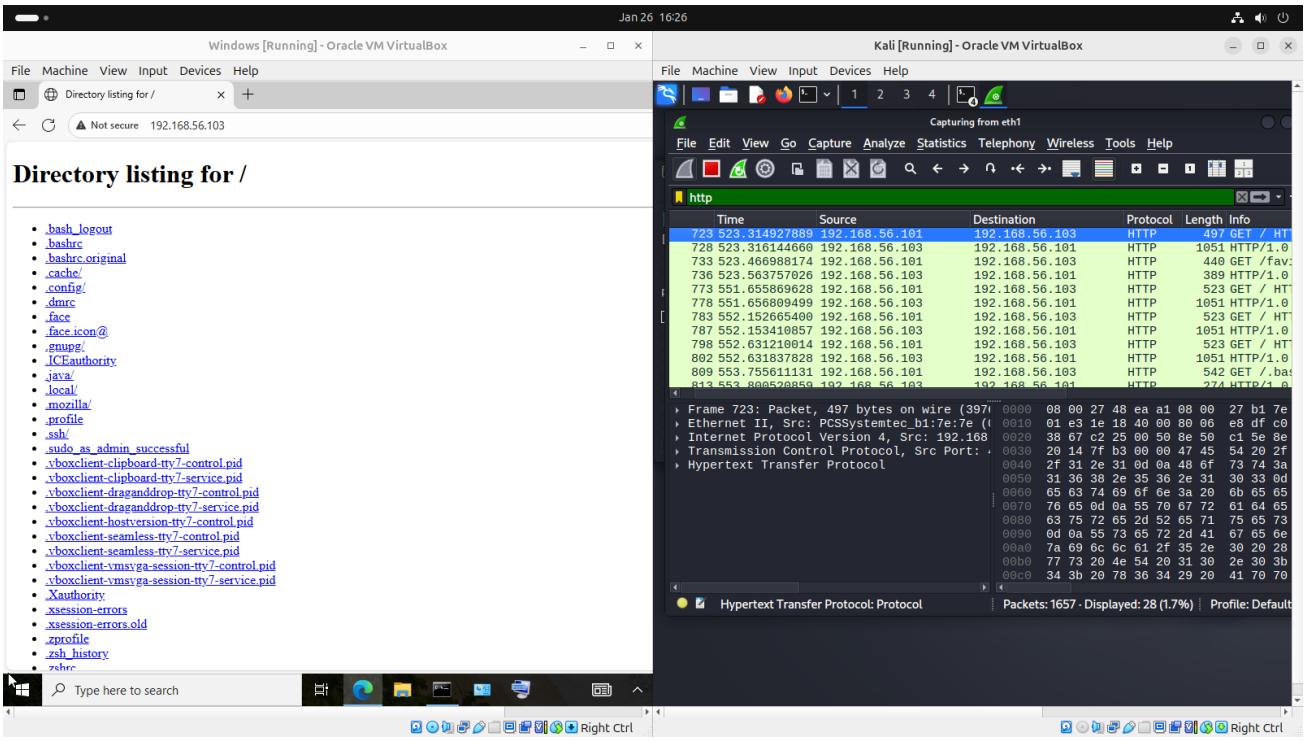



Figure 20: Access Kali HTTP Server from Windows

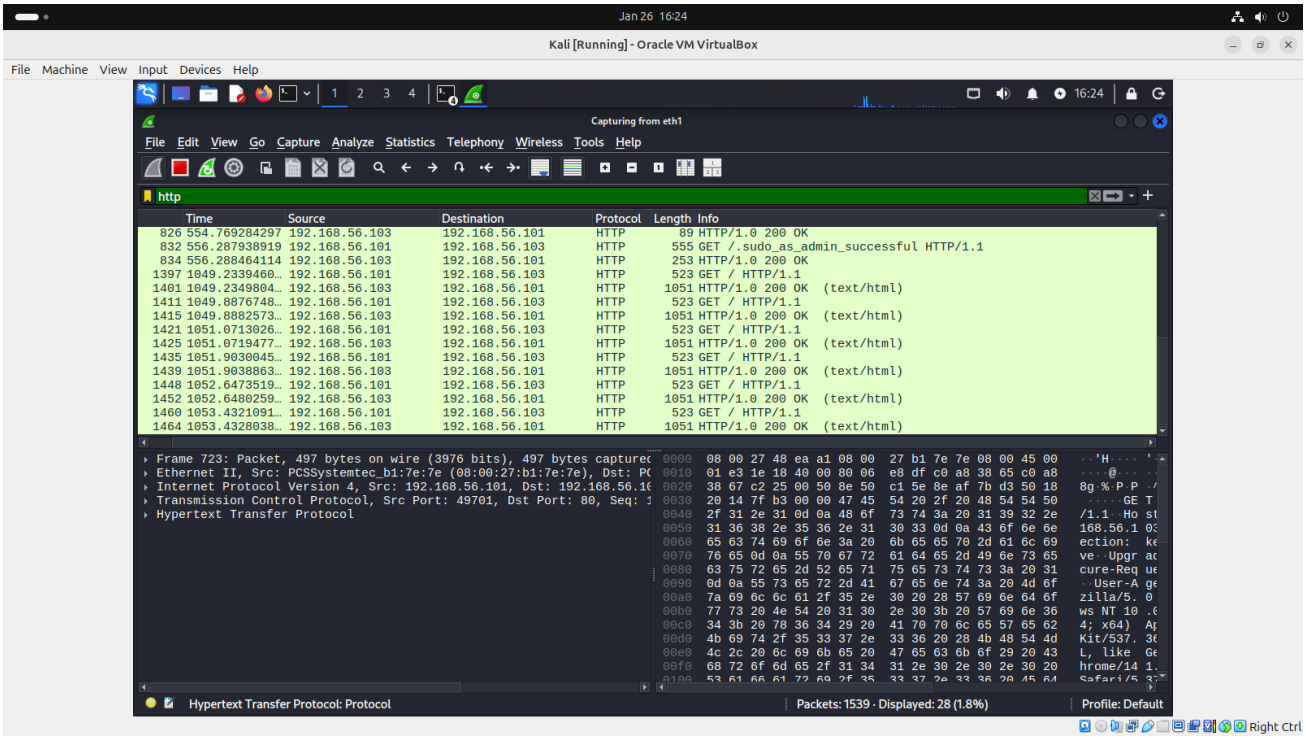



Figure 21: Capture Session Traffic

## Task 6: Presentation Layer (Layer 6) — Cleartext Data Exposure

**Vulnerability Demonstrated:**

Cleartext data transmission due to absence of encryption at the presentation layer.

**Tools Used:**

Wireshark, Web Browser, python3 http.server

**Procedure:**

A local HTTP server was started on the attacker machine using Python. The victim system accessed the server over an unencrypted HTTP connection. Network traffic was captured on the attacker system using Wireshark to analyze how data was transmitted and represented during communication.

**Observation:**

HTTP request and response data were visible in plaintext within Wireshark. Information such as requested resources, user-agent details, and response content could be decoded and read directly, indicating absence of encryption at the presentation layer.

**Impact Analysis (CIA):**

Table 8: Impact on Security Attributes

| Security Attribute | Impact |
|---|---|
| Confidentiality | Compromised |
| Integrity | Not affected |
| Availability | Not affected |

**MITRE ATT&CK Mapping:**

T1040 – Network Sniffing

**Conclusion:**

This experiment demonstrates that lack of encryption at the presentation layer allows attackers to view and decode sensitive data in transit. Secure encoding and encryption mechanisms such as HTTPS (TLS) are essential to protect confidentiality.

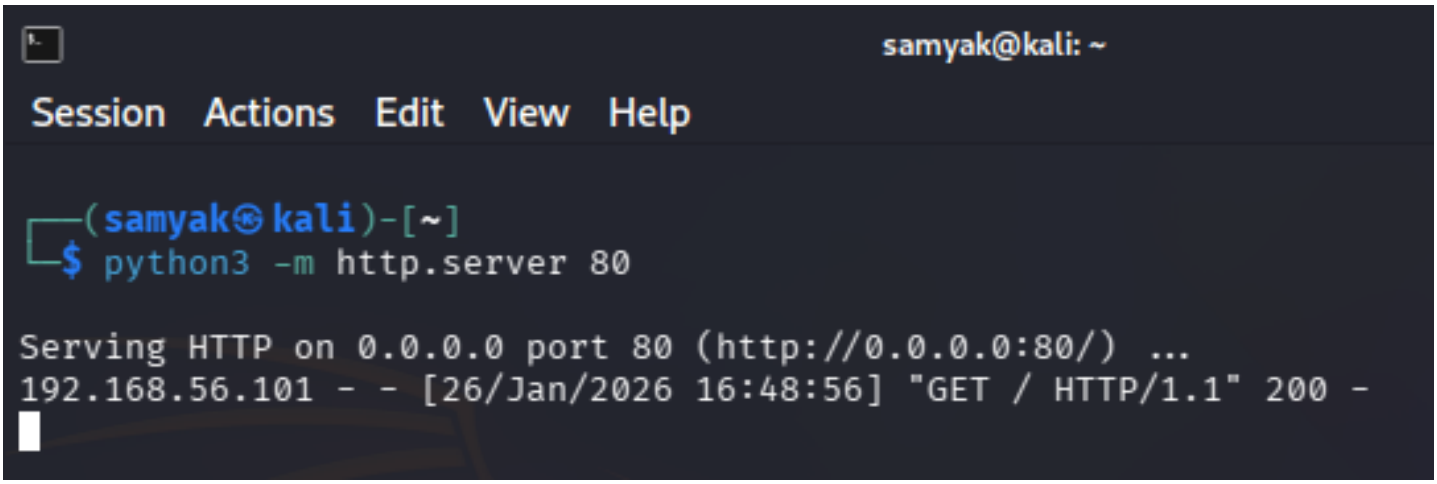



Figure 22: Python HTTP server running (Kali)

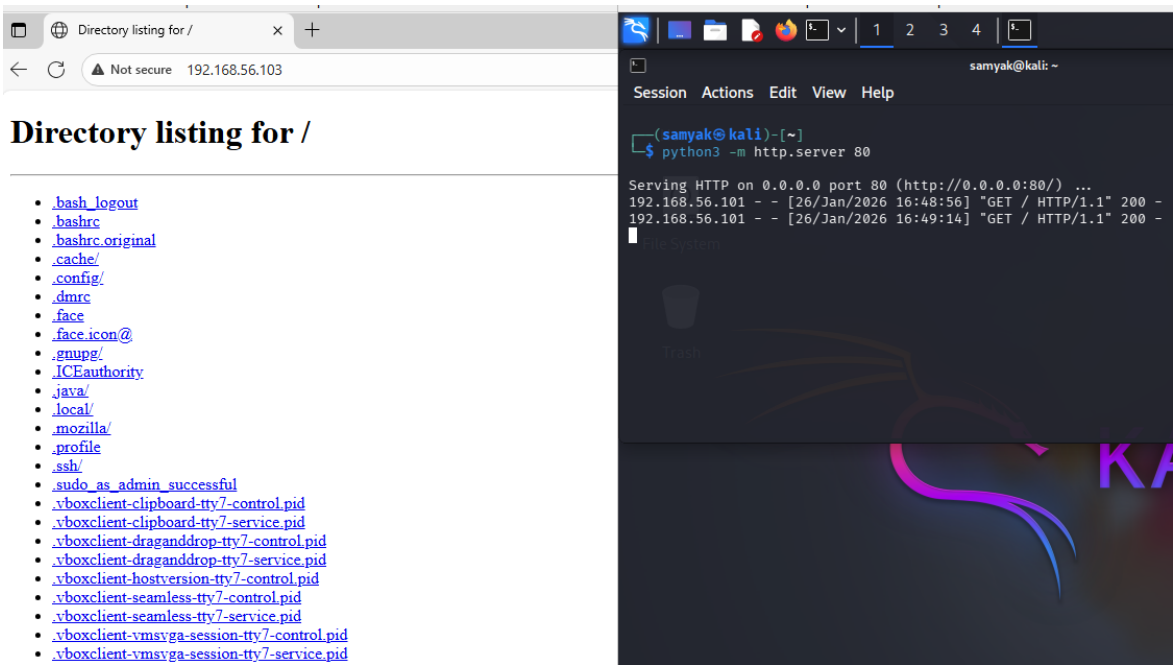



Figure 23: Windows browser accessing HTTP

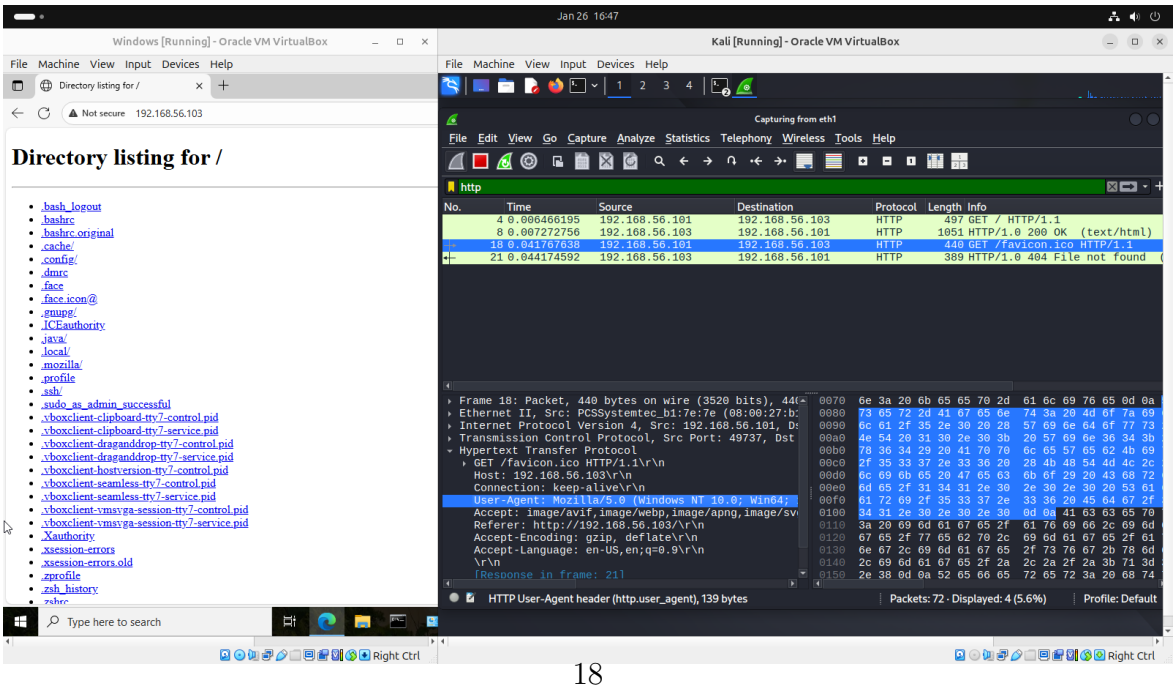





Figure 24: Observe plaintext data

## Task 7: Application Layer (Layer 7) — Credential Sniffing Attack

**Attack Performed:**

Credential sniffing targeting insecure application-layer authentication.

**Tools Used:**

dsniff, curl, python3 http.server

**Procedure:**

A simple HTTP service was hosted on the attacker machine using Python. The victim system accessed the service using HTTP Basic Authentication via the curl command. The attacker captured network traffic using the dsniff tool to extract application-level credentials.

**Observation:**

The attacker successfully captured authentication credentials transmitted by the victim. Usernames and passwords were observed in encoded (Base64) form and could be decoded directly, confirming that sensitive application-layer data was exposed.

**Impact Analysis (CIA):**

Table 9: Impact on Security Attributes

| Security Attribute | Impact |
|---|---|
| Confidentiality | Compromised |
| Integrity | Potentially compromised |
| Availability | Not affected |

**MITRE ATT&CK Mapping:**

- T1040 – Network Sniffing
- T1552 – Unsecured Credentials

**Conclusion:**

This experiment demonstrates that insecure application-layer authentication mechanisms allow attackers to intercept user credentials when unencrypted protocols are used. Secure authentication combined with encrypted communication is essential to protect application data.

```
samyak@kali: ~
Session  Actions  Edit  View  Help

┌──(samyak㉿kali)-[~]
└─$ python3 -m http.server 80
Serving HTTP on 0.0.0.0 port 80 (http://0.0.0.0:80/) ...
```

Figure 25: Start a temporary Web Server (The Listener)

```
samyak@kali: ~
Session  Actions  Edit  View  Help

┌──(samyak㉿kali)-[~]
└─$ sudo dsniff -i eth1
[sudo] password for samyak:
dsniff: magic: unknown decode: 'pop'
dsniff: magic: unknown decode: 'pop'
dsniff: listening on eth1
```

Figure 26: Start Credential Sniffer (Kali)

```
Command Prompt

C:\Users\Samyak>curl -v http://test:test@192.168.56.103
```

Figure 27: Generate Application Login Traffic

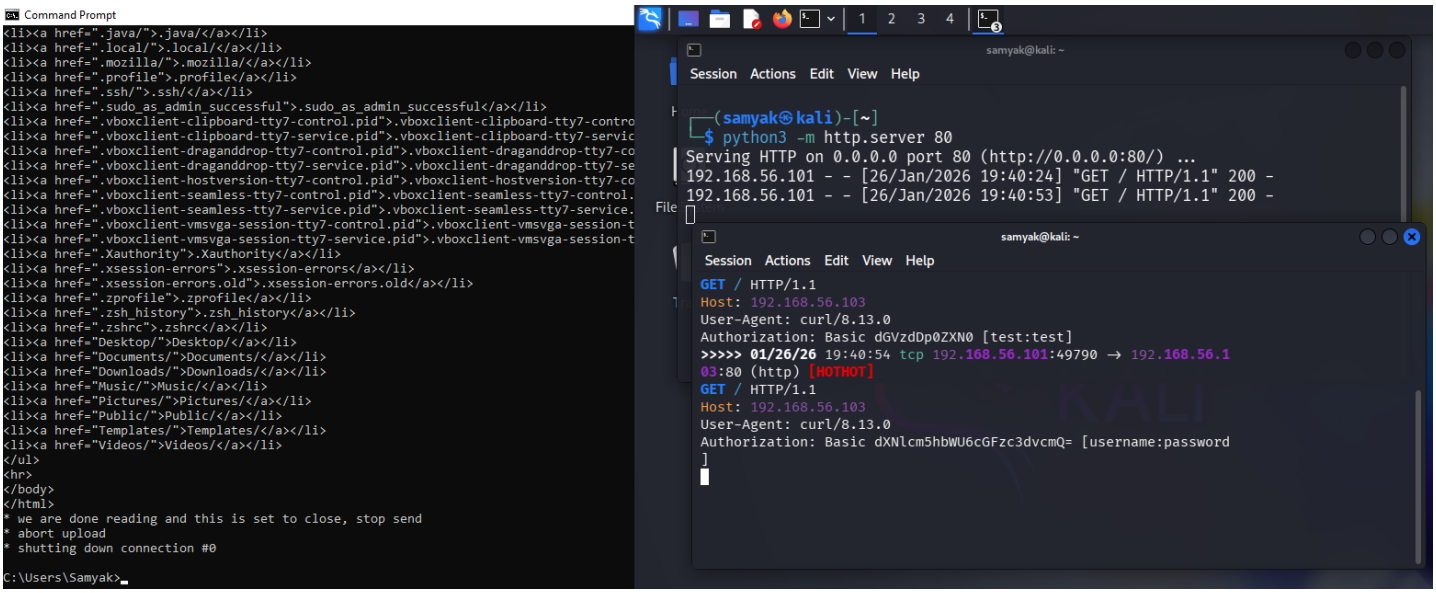



Figure 28: Observe Compromise (Kali)

## Summary Table

Table 10: OSI Layer-wise Attack Analysis

| **OSI Layer** | **Layer Name** | **Attack Performed** | **Tool(s) Used** | **Primary Impact (CIA)** |
|---|---|---|---|---|
| Layer 1 | Physical | Network interface disable (link failure simulation) | Windows Network Adapter settings | Availability |
| Layer 2 | Data Link | ARP spoofing / ARP poisoning (man-in-the-middle attack) | `arpspoof` | Confidentiality, Integrity |
| Layer 3 | Network | ICMP flood (network denial of service) | `hping3` | Availability |
| Layer 4 | Transport | TCP SYN flood | `hping3`, `netstat` | Availability |
| Layer 5 | Session | Session interception / hijacking (MITM) | `arpspoof`, `Wireshark` | Confidentiality, Integrity |
| Layer 6 | Presentation | Cleartext data exposure due to lack of encryption | `Wireshark`, HTTP | Confidentiality |
| Layer 7 | Application | Credential sniffing using HTTP basic authentication | `dsniff`, `curl`, `python3 http.server` | Confidentiality, Integrity |